

DIFERENCIAS POR PLATAFORMA Y EVIDENCIA EN IOS Y ANDROID

Estudiante / Autor	Julio Fabián Rodríguez Bazán
Repositorio GitHub	https://github.com/fabianrodriguez33/Pharma-Mobil
Sesión	Sesión 09 · Diferencias por plataforma y evidencia en iOS y Android
Rama	feature/expect-actual-rodriguez · https://github.com/fabianrodriguez33/Pharma-Mobil/tree/feature/expect-actual-rodriguez
Fecha de entrega	12/10/2026

Producto 1: Inventario de capacidades nativas

Capacidad	Declaración (commonMain)	Android (androidMain)	iOS (iosMain)
Formato de moneda	expect fun formatearSoles(valor: Double): String · commonMain/.../platform/Formato.kt	NumberFormat con Locale("es", "PE") en androidMain/.../Formato.android.kt	NSNumberFormatter con NSLocale("es_PE") en iosMain/.../Formato.ios.kt
Compartir producto	interface Compartidor (domain) + Koin · commonMain/.../domain/platform/Compartidor.kt	Intent.ACTION_SEND en androidMain/.../CompartidorAndroid.kt; registrado en PlatformModule.android.kt con androidContext()	UIActivityViewController en iosMain/.../Compartidorlos.kt; registrado en PlatformModule.ios.kt
Información del dispositivo	expect class InfoDispositivo · commonMain/.../platform/InfoDispositivo.kt	Build.VERSION.RELEASE y Build.MODEL en androidMain/.../InfoDispositivo.android.kt	UIDevice.currentDevice en iosMain/.../InfoDispositivo.ios.kt

Producto 2: Informe comparativo de diferencias

1. expect/actual frente a interfaz + Koin

En PharmaMobil usé dos estrategias. Para formatearSoles y InfoDispositivo elegí expect/actual: son funciones o clases puras, sin estado compartido ni dependencias del sistema operativo más allá de una API estática (NumberFormat, NSNumberFormatter, Build, UIDevice). El compilador exige una contrapartida por plataforma y el código común las invoca directamente, sin inyección.

Compartidor, en cambio, necesita un Context en Android para lanzar un Intent. Eso es una dependencia con ciclo de vida, imposible de expresar en un expect sin argumentos. Por eso es una interfaz en domain, con CompartidorAndroid y Compartidorlos registrados en cada platformModule de Koin. La ventaja decisiva es la testabilidad: en commonTest puedo sustituirlo por un fake que guarde el texto recibido y comprobar ProductosViewModel sin tocar ningún Intent. Una función expect no se puede sustituir así sin añadir otra capa.

En términos de diseño, expect/actual acopla el código común a una implementación fija por destino, mientras que la interfaz permite elegir la implementación en tiempo de ejecución, algo útil para pruebas, prevvisualizaciones o futuras plataformas como escritorio.

2. Qué ocurre cuando falta un actual

Para observarlo eliminé temporalmente Formato.android.kt y compilé con ./gradlew :shared:compileAndroidMain. El compilador respondió, literalmente: «e: file:///D:/DAM/pharmaMobil-master/shared/src/commonMain/kotlin/pe/edu/upeu/pharmamobil/platform/Formato.kt:7:1 Expected formatearSoles has no actual declaration in module <commonMain> for JVM» y la tarea terminó en BUILD FAILED (s09/compilador_falta_actual.log). Es un error de compilación, no de ejecución: el proyecto no genera el artefacto hasta que exista el actual. Restauré el archivo y el build volvió a pasar.

Esta es la gran ventaja de expect/actual: la omisión se detecta antes de entregar la app. Con una interfaz + Koin, en cambio, el olvido aparecería en ejecución, como NoDefinitionFoundException al resolver Compartidor. Por eso las pruebas de módulo (AppModuleTest) resultan imprescindibles para ese enfoque.

Cada source set debe aportar exactamente un actual con la misma firma, incluidos nombres y tipos de las propiedades de InfoDispositivo.

3. Contexto en Android frente a iOS

En Android, startActivity pertenece a Context. CompartidorAndroid recibe androidContext() de Koin, que es el contexto de aplicación, no de una Activity; por eso el selector se lanza con Intent.FLAG_ACTIVITY_NEW_TASK, pues sin una Activity la tarea no existe y el sistema lanzaría una excepción.

En iOS no hay Context. UIKit es un conjunto de singletons: Compartidorlos obtiene UIApplication.sharedApplication.keyWindow?.rootViewController y presenta el UIActivityViewController encima. Ese constructor sin argumentos explica por qué se registra sin parámetros en PlatformModule.ios.kt.

La consecuencia de diseño es que Android obliga a propagar una dependencia de plataforma por Koin, mientras que iOS accede directamente a la jerarquía de ventanas. Además, keyWindow puede ser nulo si no hay ventana activa, por lo que uso el operador seguro y la llamada simplemente no hace nada en ese caso.

4. Interoperabilidad Kotlin-Swift

Al exportar el framework, Kotlin se traduce así: String → String (NSString puenteado); Double → Double; suspend fun → función async/await o completion handler, según la versión de Swift; sealed class → clase base con subclases (no un enum nativo, aunque el patrón es equivalente); tipos nulables → Optional (String? → String?); Unit → Void. Por eso mantengo firmas simples en los límites de plataforma.

En Objective-C los genéricos y los tipos primitivos nulables pierden información, así que en la frontera conviene exponer tipos simples. Mis tres capacidades usan solo String y Double, lo que evita estas pérdidas.

5. Diferencias visuales al compartir

En Android, Intent.createChooser muestra una hoja inferior propia del sistema con aplicaciones destino, vista previa del texto y accesos de copiado (ver captura del Producto 4). Cada fabricante la adapta. En iOS, UIActivityViewController presenta la hoja de actividades con fila de contactos, iconos de apps y acciones como «Copiar». Ambas son nativas y el usuario las reconoce; mi código común solo entrega el texto, sin decidir su apariencia.

Una diferencia práctica: en Android el texto llega como ACTION_SEND con tipo text/plain y el sistema filtra las aplicaciones capaces de recibirlo; en iOS los activityItems se ofrecen a las extensiones de compartir instaladas. En ambos casos el usuario conserva el control final de dónde se envía el contenido.

Glosario Kotlin → Swift

Kotlin	Swift	Nota
String	String	Puenteado con NSString
Double	Double	Tipo primitivo directo
suspend fun	async/await o completion handler	Según versión de Swift
sealed class	Clase base con subclases	No se exporta como enum

Nullability (T?)	Optional (T?)	Se conserva
Unit	Void	Sin valor de retorno

Producto 3: Tercera capacidad nativa (InfoDispositivo)

commonMain: platform/InfoDispositivo.kt

```
package pe.edu.upeu.pharmamobil.platform
expect class InfoDispositivo() {
    val sistemaOperativo: String
    val versionSistema: String
    val modeloDispositivo: String
}
```

androidMain: platform/InfoDispositivo.android.kt

```
actual class InfoDispositivo actual constructor() {
    actual val sistemaOperativo: String = "Android"
    actual val versionSistema: String = Build.VERSION.RELEASE
    actual val modeloDispositivo: String = "${Build.MANUFACTURER} ${Build.MODEL}"
}
```

iosMain: platform/InfoDispositivo.ios.kt

```
actual class InfoDispositivo actual constructor() {
    actual val sistemaOperativo: String = UIDevice.currentDevice.systemName
    actual val versionSistema: String = UIDevice.currentDevice.systemVersion
    actual val modeloDispositivo: String = UIDevice.currentDevice.model
}
```

Integración en la interfaz: en presentation/productos/ProductosScreen.kt, el botón «Acerca del dispositivo» del catálogo abre un AlertDialog que muestra sistema, versión y modelo a partir de remember { InfoDispositivo() }. El único import es pe.edu.upeu.pharmamobil.platform.InfoDispositivo (paquete común).

Producto 4: Evidencias, aislamiento y README

Mensaje del compilador sin el actual: ver pregunta 2 del Producto 2 (s09/compilador_falta_actual.log).

Capturas del emulador Pixel 8 (Android 17) contra el backend PharmaSoft real (10.0.2.2:8080):

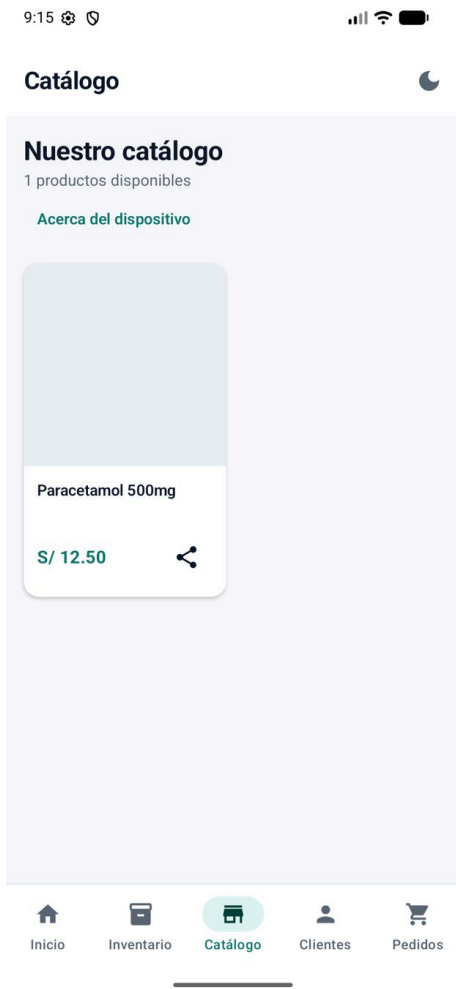


Figura 1. Catálogo con precio en soles (S/ 12.50) formateado por `formatearSoles`.

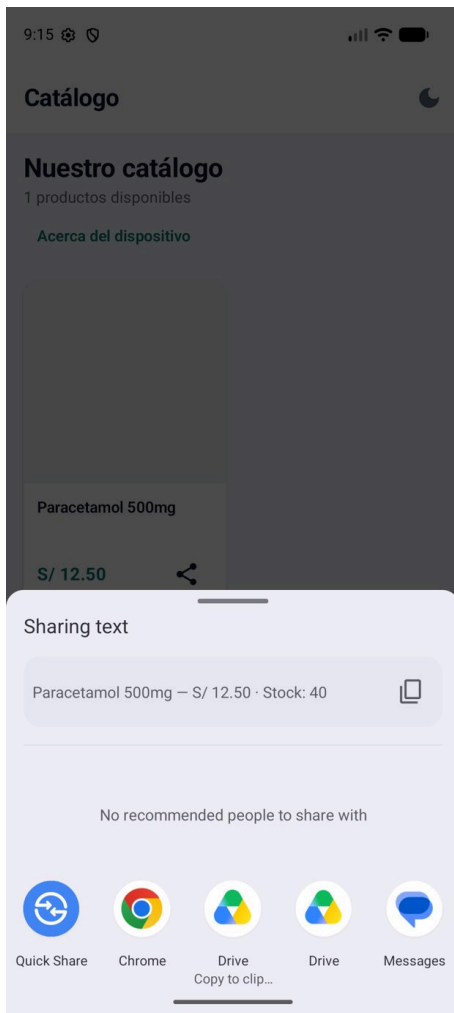


Figura 2. Selector de compartir nativo de Android (*Intent.createChooser*).

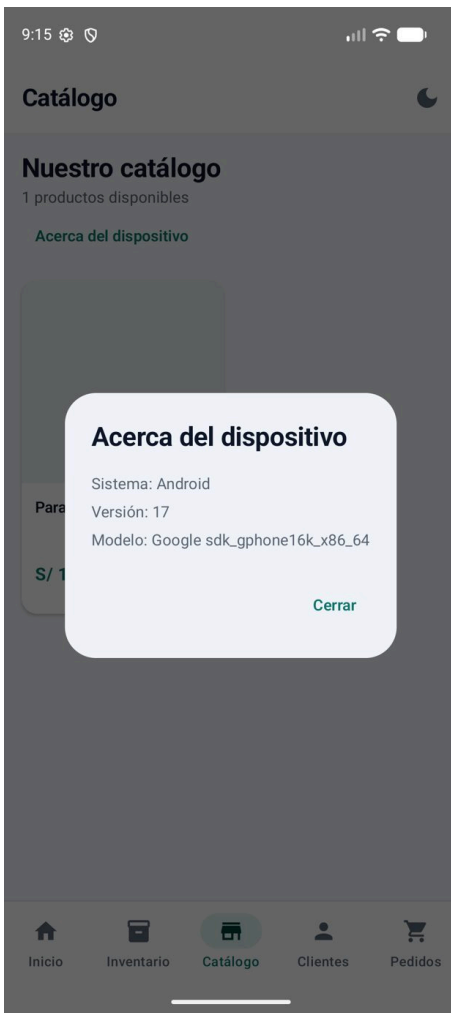


Figura 3. Tarjeta «Acerca del dispositivo» con `InfoDispositivo`.

Justificación de la ejecución en Windows/Android

La compilación de `iosMain` y el simulador de iOS requieren macOS y Xcode; el equipo de desarrollo es Windows 11, por lo que el destino iOS no se compiló ni se ejecutó. Conforme a la sección 3.4 de la guía autónoma, la evidencia en ejecución corresponde a Android, y los actual de iOS se entregan escritos y revisados contra la API de UIKit/Foundation, sin captura de simulador.

Aislamiento verificado

Búsqueda de importaciones prohibidas en `shared/src/commonMain` (`import android.*` y `import platform.UIKit.*`): 0 coincidencias. Verificación: `./gradlew testAndroidHostTest assembleDebug` → BUILD SUCCESSFUL.

README.md

Se añadió la sección «Código específico de plataforma» con las tres capacidades (formato de moneda, compartir e información del dispositivo) y su ubicación por source set.